Lineage Is Memory: Parent Inheritance and Feedback Disclosure in Short-Budget LLM-Guided Optimization

Shanglin Yang, Zhan Shi, Li Yin

Abstract

Iterative prompt and program optimizers such as GEPA refine a candidate by evaluating it on a search set, showing a proposer model a subset of failing examples, and generating a revision. Recent methods seek to improve this loop by expanding what the proposer observes in each round, using full-set error clustering, targeted minibatches, or trace inspection, or by upgrading the proposer to a larger model. Because existing evaluations always condition revisions on a surviving parent from the search lineage, the contribution of parent inheritance has remained confounded with the breadth of per-round feedback disclosure. We separate these factors in a factorial study under matched evaluation budgets across nine prompt and code optimization benchmarks and two model families.

Under short evaluation budgets where task rules must be inferred from feedback, greedy parent inheritance accounts for the dominant share of optimization progress. Accumulating edits along a parent-child lineage substantially outperforms resampling from the initial seed across both model families, even when seed-reset proposals are given a clustered summary of all search-set failures in every round, whereas upgrading the proposer to a larger model within the same family yields no comparable gain. Moreover, parent inheritance alters the value of wider per-round feedback: full-set error clustering and diverse failure selection improve accuracy when resetting to the seed and accelerate learning in the opening rounds of search, but sequential parent inheritance rapidly closes this gap as solved error categories are retained in the candidate and filtered from subsequent failure lists. On classification tasks where target labels appear only in evaluator feedback, instructing the proposer to copy revealed facts into the candidate prompt allows parent inheritance to accumulate the label inventory across rounds and surpass GEPA at less than half of its token cost, whereas the same instruction fails under seed resetting and slightly degrades accuracy when all options are already given in the seed. Finally, call-type token accounting shows that GEPA expends the vast majority of its tokens on full-set validation and minibatch gating, with only a small fraction spent on proposer reflection.

1 Introduction

Reflective prompt and program optimizers refine candidate instructions or code through an iterative propose-and-evaluate loop. In GEPA (Agrawal et al., 2025), the system runs a candidate instruction on a small minibatch, presents the inputs, outputs, and evaluator feedback to a proposer model, accepts the revision if it improves minibatch performance, and selects subsequent parents from a Pareto front. Similar evolutionary loops underlie OPRO (Yang et al., 2023), EvoPrompt (Guo et al., 2023), Promptbreeder (Fernando et al., 2023), FunSearch (Romera-Paredes et al., 2023), and AlphaEvolve (Novikov et al., 2025).

Recent work has focused on widening what the proposer observes during reflection. Liu et al. (2026) note via a coupon-collector calculation that a random minibatch of three examples exposes only a small fraction of error modes per round, and propose ESPO, which clusters all training errors into three to seven diagnostic patterns before generating edits. RPT (Bayat et al., 2026) provides a tool that aggregates critiques across the training set, AdaGEPA (Chen et al., 2026) steers minibatch slots toward identified weaknesses, Contrastive Reflection (Koh et al., 2026) pairs failing and passing examples from decision-tree slices, and CASD (Singh et al., 2026) lets a coding agent inspect the full trace corpus. In all of these iterative studies, revisions are tested only inside lineage-preserving search, where every proposal edits a surviving parent. Without a lineage-off control that resets each proposal to the initial seed, existing evaluations cannot tell how much progress comes from wider single-round reflection and how much comes from accumulating edits along a parent-child lineage.

To separate these factors, we organize an optimizer around three design choices: lineage P, feedback disclosure F, and evaluation scheduling A. Lineage P determines whether each proposal edits the initial seed $P = \text{seed}$,

which corresponds to best-of-N resampling from the seed's feedback, or edits the current search-best candidate P = parent. Disclosure F determines whether the evaluator shows the proposer only the scalar score, the first k failures, k structurally diverse failures, a random mixed minibatch of k passing and failing examples, or an LLM-clustered summary of all failures with or without expected answers. Scheduling A governs candidate evaluation and stopping. We run a crossed $P \times F$ factorial experiment under a fixed full-search-set scheduler with early stopping, complemented by a system-level comparison against GEPA under matched maximum task-model call caps. Across 1,750 runs on nine task families, comprising five constructed rule-discovery tasks, a rubric-judged customer support task, ScoNe (She et al., 2023), a 12-intent subset of Banking77 (Casanueva et al., 2020), and BIG-Bench Hard salient translation error detection (Suzgun et al., 2022), we replicate the main contrasts on `claude-haiku-5-5` and test a larger proposer model `gpt-4.1`.

Our main findings are as follows:

- **Parent inheritance outweighs single-round disclosure breadth and proposer size (Sections 4.1–4.2):** On tasks where rules must be recovered from feedback, switching from P = seed to P = parent produces the largest accuracy gain in our study, adding +0.238 across all seven primary tasks on `gpt-4.1-mini` and +0.222 across five tasks on `claude-haiku-5-5`. This advantage holds even when seed-reset proposals draw fresh random examples each round in `seed-minibatch` or receive a clustered summary of all search-set failures in every round in `seed-clustergold`: showing just four raw failures under parent inheritance still beats `seed-clustergold` by +0.157. Upgrading the proposer from `gpt-4.1-mini` to `gpt-4.1` on three feedback-dependent prompt tasks yields no clear gain in either row, while parent inheritance under `gpt-4.1` adds +0.380.
- **Fact retention in the prompt explains the Banking77 gap with GEPA (Section 4.3):** On the 12-intent Banking77 task, where intent names appear only in error feedback, baseline `parent-first` initially scored 0.372 against 0.664 for GEPA. Inspection of the search logs showed that early formatting-only edits scored 0.000 and stayed anchored to the seed, whereas GEPA's reflection prompt explicitly tells the proposer to copy domain facts from feedback into the instruction. In a follow-up experiment on Banking77 and BBH-STE, appending one sentence with this instruction raised `parent-first` on Banking77 from 0.372 to 0.856 at 43% of GEPA's median token cost, while `seed-first` with the same sentence retained only the 4 intents shown in round 1 and scored 0.317. On BBH-STE, where all six option labels are already in the seed, the same sentence reduced accuracy by −0.061.
- **Parent inheritance closes the gap with wider per-round disclosure over multiple rounds (Sections 4.4–4.6):** When resetting to the seed, showing only the first four failures crowds out zero-feedback code priors on unshown error categories, and both diverse sampling and target-preserving clustering improve accuracy over `first-k` disclosure. Under parent inheritance, full-set clustering initially leads in round 2 by +0.173, but as solved categories are retained in the parent and filtered from future failure lists, `first-k` disclosure shifts across 46% to 86% of the search set, catching up by round 4 and finishing at the same level. Conversely, random mixed minibatches reduce parent-inheritance accuracy by spending feedback slots on passing examples.
- **GEPA spends most of its token budget on validation and minibatch gating (Section 4.7):** Across all seven prompt tasks evaluated with GEPA, proposer reflection consumes only 2% to 15% of total tokens, while full search-set validations and minibatch checks consume the remaining 85% to 98%.

2 Related Work

2.1 LLM-Guided Program Evolution

Evolutionary search with language models, as in FunSearch (Romera-Paredes et al., 2023), AlphaEvolve (Novikov et al., 2025), and ImprovEvolve (Kravatskiy et al., 2026), maintains an archive of high-scoring programs and prompts a model to mutate or recombine selected parents. Performance gains in this literature are typically credited to the evolutionary loop as a whole. The closest methodological study is Evolution or Illusion? (Oved et al., 2026), which varies how a fixed generation budget is split between initial seeds and evolutionary generations, showing that strategy rankings can reverse across budget scales. Oved et al. (2026) hold feedback presentation fixed, do not break down token usage by call type, and treat best-of-N resampling only as a reference curve. We complement their work by fixing the evaluation budget cap and crossing parent inheritance with feedback disclosure.

2.2 Reflective Prompt Optimization

Early automatic prompt optimizers either backpropagated textual critiques from failing examples, as in ProTeGi (Pryzant et al., 2023) and TextGrad (Yuksekgonul et al., 2024), or showed the proposer a score-ordered history of past instructions, as in OPRO (Yang et al., 2023) and EvoPrompt (Guo et al., 2023). GEPA (Agrawal et al., 2025) combined reflection on a small random minibatch of default size $b = 3$ with Pareto-front parent selection and minibatch acceptance gating. GEPA 0.0.7 also includes a prompt instruction asking the reflection model to copy domain-specific facts from feedback into the revised prompt; Section 4.3 tests this instruction directly. Other recent studies examine when reflective optimization helps or fails: Lumer et al. (2026) show that per-dimension feedback transfers better than scalar scores under noisy judges; Giummarra et al. (2026) find that GEPA improves MATH-500 accuracy but fails to improve on HotpotQA with local reasoning models while increasing token cost; and Ma et al. (2024) show that prompt optimizers rely heavily on the proposer's pre-trained task prior, matching our results on `dates` and `scone`, where the model prior already fits the specification.

2.3 Expanding Reflection Scope

Several recent papers argue that small random minibatches under-sample the error distribution during reflection. Liu et al. (2026) formalize this via a coupon-collector bound and introduce ESPO, which clusters all training failures into three to seven error patterns in a Diagnose stage before generating pattern-specific edits in a Diversify stage and bootstrap-selecting among them in a Stabilize stage. RPT (Bayat et al., 2026) gives the optimizer a clustering tool over full-set critiques; AdaGEPA (Chen et al., 2026) swaps minibatch slots toward identified failure modes; CASD (Singh et al., 2026) replaces iterative search with a coding agent that reads all traces before writing a single prompt; and Contrastive Reflection (Koh et al., 2026) pairs failing and passing examples from decision-tree slices. Every iterative method in this group is evaluated only inside a lineage-preserving loop, without a seed-reset control or a zero-feedback baseline. Testing disclosure schemes with and without parent inheritance shows that single-round error coverage strongly limits seed-reset optimization, whereas under parent inheritance its terminal effect is much smaller.

2.4 Validation Efficiency and Example Selection

Explore More, Validate Less (Zhou and Cheng, 2026) notes that GEPA wastes evaluations on minibatches where the parent already scores perfectly, producing no edit, and proposes trimming full-set validations. Section 4.7 extends this by decomposing GEPA's token budget across reflection, minibatch gating, and full validation. Choosing a small representative set of examples to specify program behavior is also a classic problem in inductive program synthesis (Pu et al., 2017); our `diverse` condition adapts farthest-point input selection to iterative error feedback.

3 Methodology and Experimental Design

3.1 The P/F/A Optimization Loop

We study iterative optimization over a candidate space $\mathcal{C}$ consisting of natural language system prompts or Python functions. Each task defines a search set $\mathcal{D}_{\text{search}} = \{(x_i, y_i^*)\}_{i=1}^{N_{\text{search}}}$ partitioned into G failure categories $\{C_1^{\text{search}}, \ldots, C_G^{\text{search}}\}$, and a disjoint held-out test set $\mathcal{D}_{\text{holdout}} = \{(x'_j, y'^*_j)\}_{j=1}^{N_{\text{holdout}}}$ partitioned into $\{C_1^{\text{holdout}}, \ldots, C_G^{\text{holdout}}\}$ that is never accessed during search. An evaluator $E(c, \mathcal{D})$ runs candidate $c \in \mathcal{C}$ on $\mathcal{D}$, returning binary instance scores $s_i(c) \in \{0, 1\}$, accuracy $S(c, \mathcal{D}) = \frac{1}{|\mathcal{D}|} \sum_i s_i(c)$, and failing indices $\mathcal{E}(c) = \{i : s_i(c) = 0\}$.

Algorithm 1: Controlled P/F/A Optimization Loop

1. **Input:** Seed candidate $c_0 \in \mathcal{C}$, evaluation budget cap B, lineage policy $P \in \{\text{seed}, \text{parent}\}$, disclosure operator F, search set $\mathcal{D}_{\text{search}}$, held-out set $\mathcal{D}_{\text{holdout}}$.
2. **Initialize:** Evaluate c_0 on $\mathcal{D}_{\text{search}}$ and cache $E(c_0, \mathcal{D}_{\text{search}}) = \big(S(c_0, \mathcal{D}_{\text{search}}), \mathcal{E}(c_0)\big)$. Set history $\mathcal{H}_0 = \{c_0\}$ and current best $c_0^* = c_0$.
3. **For** round $t = 1, \ldots, B - 1$ **do:**
   - **Stop check (A):** If $S(c_{t-1}^*, \mathcal{D}_{\text{search}}) = 1.0$, stop early.
   - **Parent selection (P):** Set $p_t = c_0$ if $P = \text{seed}$, or $p_t = c_{t-1}^*$ if $P = \text{parent}$. Retrieve cached $E(p_t, \mathcal{D}_{\text{search}})$.

- **Feedback disclosure (F):** Build feedback string $f_t = F(p_t, \mathcal{D}_{\text{search}}, k = 4)$ from $E(p_t, \mathcal{D}_{\text{search}})$.
- **Proposal:** Query proposer M_{prop} with `PROPOSE_TEMPLATE` (Appendix B.1) using the task objective, p_t, and f_t to generate c_t.
- **Evaluation (A):** Evaluate c_t on $\mathcal{D}_{\text{search}}$, consuming 1 evaluation from cap B, and cache $E(c_t, \mathcal{D}_{\text{search}})$. If $S(c_t, \mathcal{D}_{\text{search}}) > S(c^*_{t-1}, \mathcal{D}_{\text{search}})$, set $c^*_t = c_t$; otherwise keep $c^*_t = c^*_{t-1}$ under strict improvement tie-breaking.

4. **Output:** Evaluate c^* once on $\mathcal{D}_{\text{holdout}}$ and return held-out accuracy $Y = S(c^*, \mathcal{D}_{\text{holdout}})$.

Factor 1: Lineage (P). Under $P =$ seed, every round sets $p_t = c_0$ and reuses the cached evaluation of c_0. Under deterministic disclosure schemes such as `none`, `first`, and `diverse`, both p_t and f_t stay fixed across rounds, so the $B - 1$ proposals are independent samples from M_{prop} at default provider temperature conditioned on c_0. Under `minibatch`, `cluster`, and `clustergold`, f_t varies across rounds while still requiring a single-edit jump from c_0. Under $P =$ parent, every round edits the current search-best candidate $p_t = c^*_{t-1}$, reusing its cached evaluation if the previous proposal did not strictly improve on it. When $S(c_0, \mathcal{D}_{\text{search}}) = 0.0$, as on `labels`, `routing`, `replies`, and `banking77`, strict-improvement tie-breaking keeps $p_t = c_0$ until a proposal scores above zero; this occurs in round 1 on `routing` and `replies` and in round 1 or 2 on `labels`, but can take several rounds on `banking77` (Section 4.3). Once c^*_t improves over c_0, $P =$ parent combines three mechanisms relative to $P =$ seed: text inheritance, which keeps discovered rules in p_{t+1}; residual-error exposure shift, which removes solved cases from $\mathcal{E}(p_{t+1})$ so new categories enter the $k = 4$ display window; and multi-step edit decomposition, which allows the proposer to fix a subset of categories per round. Secondary inspiration candidates are disabled in all cells.

Factor 2: Feedback Disclosure (F). Failing examples are formatted with their input, wrong output, and expected answer or grading rubric (Appendix B). We compare six operators: - `none`: Masks the evaluation of p_t by setting `score=None`, omitting instance scores, and rendering `{{ feedback }}` as `(no feedback)`. Because $p_t = c_0$ carries no round-dependent signal under `none`, `seed-none` is the shared zero-feedback baseline. - `first`: Shows $S(p_t, \mathcal{D}_{\text{search}})$, per-instance binary scores, and the first $k = 4$ failures of p_t in dataset order, followed by the count of remaining failures. In round 1, this exposes 2 categories on `labels`, `dates`, `replies`, and `size`, up to 3 on `duration`, and up to 4 on `routing`, `banking77`, and `bbh_ste`. - `diverse`: Selects $k = 4$ failures via greedy farthest-point sampling under normalized Levenshtein distance over a structural input signature defined by `_shape` in `presets.py`, which maps lowercase letters to `a`, uppercase letters to `A`, and digits to `9` while preserving punctuation, starting from the first failure. - `minibatch`: Draws $k = 4$ examples uniformly at random without replacement from $\mathcal{D}_{\text{search}}$ in each round using `random.Random(1000 + r)` for run seed $r \in \{0, \ldots, 9\}$, and shows whether p_t passed or failed each drawn example alongside its expected answer, while still gating acceptance on the full search set. - `cluster`: Passes all failures $\mathcal{E}(p_t)$ to one call of the clusterer model, which matches M_{prop} and groups them into 3 to 7 patterns, outputting one line per pattern with a description, failure count, and one verbatim representative wrong answer (`CLUSTER_PROMPT`, Appendix B.3). Clustering tokens are included in search tokens. - `clustergold`: Identical to `cluster`, except the prompt requires each representative failure to include the input, wrong answer, and full expected answer (`CLUSTER_GOLD_PROMPT`, Appendix B.3).

Scheduling (A) and GEPA Comparison. In our $P \times F$ grid, every proposal is evaluated on all of $\mathcal{D}_{\text{search}}$ up to $B = 8$ evaluations for prompt tasks, corresponding to c_0 plus 7 proposals and at most $8 \times N_{\text{search}}$ task calls, and $B = 12$ evaluations for code tasks, corresponding to c_0 plus 11 proposals executed with a 60-second subprocess timeout across 24 test cases for at most 288 executions. Missing fenced blocks trigger at most one retry, though all 1,750 runs produced valid blocks on the first try; code exceptions score 0.0 with the traceback included in feedback. Proposer, task, and clusterer models use default provider sampling in `openai` SDK 2.44.0 for `gpt-4.1-mini` and `gpt-4.1` and `anthropic` SDK 0.112.0 with default adaptive thinking for `claude-haiku-5-5`, while the `replies` judge uses `gpt-4.1-mini` at `temperature=0` with response caching. For system-level comparison, we run `gepa` 0.0.7 (Agrawal et al., 2025) with default minibatch size $b = 3$, Pareto selection, minibatch acceptance gating, its built-in reflection prompt, and the same task-call cap max_metric_calls $= 8 \times N_{\text{search}}$, testing both unmodified `GEPA mb3` and early-stopped `GEPA mb3 + stop`.

3.2 Benchmark Suite and Task Specifications

Table 1 summarizes the seven primary tasks and two public extension tasks.

Task Family	Artifact Type	Categories (G)	$\lvert\mathcal{D}_{\text{search}}\rvert$	$\lvert\mathcal{D}_{\text{holdout}}\rvert$	Budget Cap (B)	Max Task Calls	Seed Score (Search / Holdout)	Latent Specification Withheld from c_0 and Revealed via Feedback
`labels`	Prompt	12	24 (2/cat)	24 (2/cat)	8	192	0.00 / 0.00	Exact `snake_case` category names (`ipv4`, `hex_color`, `us_date`, `semver`, etc.)
`dates`	Prompt	12	24 (2/cat)	24 (2/cat)	8	192	0.75 / 0.75	Slash vs. dot day/month order, 2-digit century cutoff (19xx vs. 20xx), `invalid` token
`routing`	Prompt	6	16 (2–3/cat)	12 (2/cat)	8	128	0.00 / 0.00	6 queue names (`payments`, `fulfillment`, `logistics`, `trust_safety`, `privacy`, `care`) & edge policies
`replies`	Prompt (Free-text)	8	16 (2/cat)	16 (2/cat)	8	128	0.00 / 0.00	8 Acme support policies (refund window, >7-day $10 credit threshold, URLs, legal email, GDPR rules)
`scone`	Prompt (Public NLI)	6	36 (6/cat)	72 (12/cat)	8	288	0.89 / 0.89	6 negation-scope conditions from ScoNe (She et al., 2023); binary `entailment` / `neutral` labels
`duration`	Code (Python)	7	24	24	12	288	0.46 / 0.42	Plain seconds, compound units (`1h 30m`), English words, clock (`M:SS`, `HH:MM:SS`), ISO 8601 (`PT1H30M`), `ValueError`
`size`	Code (Python)	7	24	24	12	288	0.21 / 0.21	Plain bytes, SI decimal (`10KB`), IEC binary (`4MiB`, `2Gi`), single-letter (`4K`), comma separators (`1,024`), English words
`banking77`	Prompt (Public Cls.)	12	24 (2/cat)	36 (3/cat)	8	192	0.00 / 0.00	12 intent labels sampled from Banking77 (Casanueva et al., 2020), withheld from the initial seed
`bbh_ste`	Prompt (Public QA)	6	18 (3/cat)	36 (6/cat)	8	144	0.56 / 0.56	6 translation error classes in BBH-STE (Suzgun et al., 2022); all 6 option names given in seed

Table 1: Specifications of the seven primary tasks and two public extension tasks.

In the five constructed tasks, namely `labels`, `dates`, `routing`, `duration`, and `size`, as well as in `replies`, the seed c_0 omits key rules or label names so that high accuracy requires learning from feedback (Appendix B.2). For the two code tasks, c_0 is a 4-line function that handles only single-character suffixes: `s`, `m`, and `h` on `duration`, and `B`, `K`, `M`, and `G` on `size`. This setup favors methods that accumulate rules across rounds. To test public benchmarks, we included `scone` (She et al., 2023) in Batch A.1 and added `banking77` (Casanueva et al., 2020) and `bbh_ste` (Suzgun et al., 2022) in Batch A.4. For `banking77`, 12 of 77 intents were sampled with `random.Random(0)` before registration after an excluded 20-intent smoke test scored at most 0.15 within 8 evaluations, drawing 2 search and 3 held-out queries per intent. For `bbh_ste`, the six option names were included in c_0, and examples exceeding the 300-character feedback truncation limit were filtered before registration, retaining 198 of 250 examples and sampling 3 search and 6 held-out examples per class.

3.3 Statistical Estimands, Bootstrap Protocol, and Run Accounting

The replication unit is an optimization run with seed $r \in \{0, \ldots, 9\}$, giving $n = 10$ runs per cell. Let $Y_{\tau,P,F,r} \in [0,1]$ denote held-out accuracy $S(c^*, \mathcal{D}_{\text{holdout}})$ on task $\tau \in \mathcal{T}$, reserving t for optimization rounds, and let $\bar{Y}_{\tau,P,F} = \frac{1}{10}\sum_{r=0}^{9} Y_{\tau,P,F,r}$ denote the cell mean. Weighting tasks in $\mathcal{T}$ equally, we estimate three quantities:

1. **Lineage main effect (Δ_{lineage}):**

$$\Delta_{\text{lineage}} = \frac{1}{|\mathcal{T}| \cdot |\mathcal{F}|} \sum_{\tau \in \mathcal{T}} \sum_{F \in \mathcal{F}} \left(\bar{Y}_{\tau,\text{parent},F} - \bar{Y}_{\tau,\text{seed},F}\right).$$

2. **Attenuation interaction (I_X):** For $X \in \{\text{diverse}, \text{minibatch}, \text{cluster}, \text{clustergold}\}$ and $\delta_{\tau,P}(X) = \bar{Y}_{\tau,P,X} - \bar{Y}_{\tau,P,\text{first}}$,

$$I_X = \frac{1}{|\mathcal{T}|} \sum_{\tau \in \mathcal{T}} \left(\delta_{\tau,\text{seed}}(X) - \delta_{\tau,\text{parent}}(X)\right).$$

   A positive $I_X > 0$ means disclosure X improves over `first` more under seed resetting than under parent inheritance.

3. **Unshown-category proposal coverage ($\text{Cov}_{\text{unshown}}$):** In the `seed` row, $\mathcal{U}_\tau \subset \{1, \ldots, G\}$ is the set of categories where c_0 fails at least one search example in $> 50\%$ of `seed` runs, excluding categories shown among the first four failures in $> 50\%$ of `seed-first` runs, with `seed-none` using the same reference set $\mathcal{U}_\tau$. Across proposals $\mathcal{V}_r = \{c_1, \ldots, c_{m_r}\}$ in run r,

$$\text{Cov}_{\text{unshown}}(r) = \frac{1}{|\mathcal{U}_\tau|} \sum_{g \in \mathcal{U}_\tau} \frac{1}{|\mathcal{V}_r|} \sum_{c \in \mathcal{V}_r} \mathbf{1}\left[s_i(c) = 1 \text{ for all } i \in C_g^{\text{eval}}\right],$$

   where C_g^{eval} is C_g^{search} for prompt tasks and C_g^{holdout}, evaluated offline after search completes, for code tasks. Crowding-out ($\mathcal{H}_{C1}$) holds when $\bar{\text{Cov}}_{\text{unshown}}(\text{seed-first}) - \bar{\text{Cov}}_{\text{unshown}}(\text{seed-none}) < 0$.

Bootstrap inference and run accounting. All confidence intervals are 95% percentile bootstrap intervals given by the 1-indexed order statistics $[\tilde{\psi}_{(251)}, \tilde{\psi}_{(9750)}]$ over $N_{\text{boot}} = 10{,}000$ task-stratified resamples initialized with `random.Random(0)` in `results/analyze.py`, resampling 10 runs with replacement within each task-by-condition cell. For conditional tokens-to-solve ratios ($\mathcal{H}_{C3}$), solved runs are resampled within each arm on tasks with at least 3/10 solved runs per arm. Alongside these fixed-strata intervals, we report per-task intervals (Table 2A), leave-one-task-out intervals (Table 8), and two-stage task-cluster bootstrap intervals that resample tasks at the outer level: on `gpt-4.1-mini`, the 7-task cluster interval is $[+0.104, +0.363]$ for Δ_{lineage} and $[+0.052, +0.336]$ for $I_{\text{clustergold}}$, which is positive on all 7 tasks with exact two-sided sign test $p = 0.0156$. All 1,750 completed runs in `results/runs.jsonl` come from five sequential pre-registered batches documented in Table 7 and Appendix A: Batch A.1 with 750 runs, Batch A.2 with 140 runs, Batch A.3 with 350 runs, Batch A.4 with 470 runs, and Batch A.5 with 40 runs.

4 Experimental Results

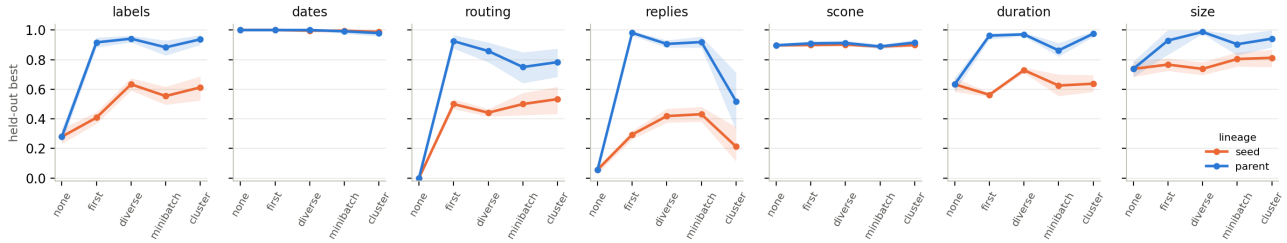


Figure 1: Held-out accuracy of the search-best candidate across disclosure schemes on the seven primary tasks for `gpt-4.1-mini`, comparing seed-reset resampling (P = seed, orange) against greedy parent inheritance (P = parent, blue). Points show means over $n = 10$ runs per cell; shaded bands indicate 95% bootstrap confidence intervals. The zero-feedback baseline (`none`) is shared by both rows.

4.1 Parent Inheritance Across Task Families and Models

Tables 2A and 2B report held-out accuracy across the seven primary tasks on `gpt-4.1-mini`.

Lineage (P) & Disclosure (F)	`labels`	`dates`	`routing`	`replies`	`scone`	`duration`	`size`	Mean (All 7)	Mean (5 Non-Ceil.)
`seed-none` (Zero feedback)	0.279	1.000	0.000	0.056	0.897	0.633	0.738	0.515	0.341
`seed-first`	0.408	1.000	0.500	0.294	0.899	0.562	0.767	0.633	0.506
`seed-diverse`	0.633	0.996	0.442	0.419	0.901	0.729	0.738	0.694	0.592
`seed-minibatch`	0.554	0.996	0.500	0.431	0.889	0.625	0.804	0.686	0.583
`seed-cluster`	0.613	0.988	0.533	0.212	0.899	0.637	0.812	0.671	0.561
`seed-clustergold` (Batch A.2)	0.683	1.000	0.625	0.794	0.908	0.637	0.875	0.789	0.723
`parent-first`	0.917	1.000	0.925	0.981	0.910	0.963	0.929	0.946	0.943
`parent-diverse`	0.942	1.000	0.858	0.906	0.912	0.971	0.988	0.940	0.933
`parent-minibatch`	0.883	0.992	0.750	0.919	0.889	0.863	0.904	0.886	0.864
`parent-cluster`	0.938	0.979	0.783	0.519	0.915	0.975	0.942	0.864	0.831
`parent-clustergold` (Batch A.2)	0.958	0.996	0.842	0.869	0.900	0.971	0.963	0.928	0.921
Δ_{lineage} (`first`, `diverse`, `minibatch`, `cluster`)	**+0.368**	−0.002	**+0.335**	**+0.492**	+0.010	**+0.304**	**+0.160**	**+0.238**	**+0.332**
95% Bootstrap CI for Δ_{lineage}	[+0.334, +0.402]	[−0.009, +0.004]	[+0.283, +0.387]	[+0.428, +0.552]	[+0.000, +0.020]	[+0.277, +0.330]	[+0.121, +0.199]	[+0.224, +0.252]	[+0.312, +0.351]

Table 2A: Held-out accuracy cell means and per-task lineage effect Δ_{lineage} on `gpt-4.1-mini` ($n = 10$ runs per cell). "5 Non-Ceil." averages over the five tasks with headroom above `seed-none` (`labels`, `routing`, `replies`, `duration`, `size`), excluding `dates` and `scone`.

Cross-Task Contrast (7 Tasks Equal-Weighted, `gpt-4.1-mini`)	Point Estimate & 95% Bootstrap CI	5 Non-Ceiling Tasks Mean
Lineage main effect Δ_{lineage} (`first`, `diverse`, `minibatch`, `cluster`)	**+0.238** [+0.224, +0.252]	**+0.332** [+0.312, +0.351]
Lineage main effect across all 5 schemes / across `clustergold` only	+0.218 [+0.204, +0.233] / +0.139 [+0.096, +0.181]	+0.305 / +0.198
Cross-row contrast: `parent-first` − `seed-clustergold` / `parent-first` − `seed-minibatch`	**+0.157** [+0.124, +0.195] / **+0.261** [+0.235, +0.285]	+0.220 / +0.360
Attenuation interaction I_{diverse} = (diverse − first)$_{\text{seed}}$ − (diverse − first)$_{\text{parent}}$ ($\mathcal{H}_{C2}$)	**+0.068** [+0.043, +0.092]	+0.096
Attenuation interaction $I_{\text{minibatch}}$ = (minibatch − first)$_{\text{seed}}$ − (minibatch − first)$_{\text{parent}}$	**+0.113** [+0.079, +0.148]	+0.156
Attenuation interaction I_{cluster} = (cluster − first)$_{\text{seed}}$ − (cluster − first)$_{\text{parent}}$	**+0.120** [+0.073, +0.167]	+0.167
Attenuation interaction $I_{\text{clustergold}}$ = (clustergold − first)$_{\text{seed}}$ − (clustergold − first)$_{\text{parent}}$	**+0.174** [+0.127, +0.222]	+0.239
Seed row within-row contrasts: `seed-diverse` − `seed-first` / `seed-clustergold` − `seed-first`	+0.061 [+0.046, +0.077] / +0.156 [+0.122, +0.188]	+0.086 / +0.217
Seed row within-row contrasts: `seed-minibatch` − `seed-first` / `seed-cluster` − `seed-first`	+0.053 [+0.030, +0.075] / +0.038 [+0.009, +0.067]	+0.077 / +0.055
Parent row within-row contrasts: `parent-diverse` − `parent-first` / `parent-clustergold` − `parent-first` ($\mathcal{H}_{S2\text{-gold}}$)	−0.007 [−0.025, +0.012] / −0.018 [−0.053, +0.010]	−0.010 / −0.022
Parent row within-row contrasts: `parent-minibatch` − `parent-first` / `parent-cluster` − `parent-first`	−0.061 [−0.086, −0.035] / −0.082 [−0.119, −0.045]	−0.079 / −0.112

Table 2B: Cross-task interaction and within-row disclosure contrasts on `gpt-4.1-mini`.

Parent inheritance produces the largest accuracy contrast in the grid, increasing 7-task held-out accuracy by $\Delta_{\text{lineage}} = +0.238$ across the four Batch A.1 disclosure schemes and by +0.218 across all five schemes (Table 2B). Importantly, this gain is not simply a consequence of `seed-first` observing the same four failures each round. Even when seed-reset proposals draw fresh random examples every round in `seed-minibatch` or receive a target-preserving summary of all search-set failures in every round in `seed-clustergold`, `parent-first` beats both by +0.261 and +0.157, and `parent-clustergold` beats `seed-clustergold` by +0.139 across all seven tasks and +0.198 across the five feedback-dependent tasks. Thus, even when the proposer sees a summary of all failing categories, rewriting the seed from scratch in a single step falls well short of accumulating edits across rounds.

Across tasks, Figure 1 and Table 2A show two distinct regimes. On `dates` and `scone`, the model's pre-trained prior already solves most cases without feedback, scoring 1.000 and 0.897 under `seed-none` and leaving little room for lineage to help. On the five tasks where rules or policies are withheld from c_0, parent inheritance adds between +0.160 on `size` and +0.492 on `replies`, averaging +0.332. Every pre-registered `parent` cell on these five tasks beats every pre-registered `seed` cell; the only crossover involves `seed-clustergold` beating `parent-cluster`

on `replies`, where the unadjusted `cluster` prompt dropped grading rubrics (Section 4.6). Replicating the experiment on `claude-haiku-5-5` across the same five tasks confirms these patterns (Table 3, Panel A): parent inheritance adds +0.222 overall and +0.257 when excluding `duration`, where Haiku starts near ceiling at 0.954, and `parent-first` again outperforms both `seed-minibatch` and `seed-clustergold`.

Model Configuration & Cell	`labels`	`routing`	`replies`	`duration`	`size`	Mean (5 Tasks)	Mean (3 Prompt Tasks)
Panel A: `claude-haiku-5-5` Replication ($n = 10$ runs per cell, 450 runs across Batches A.3 & A.4)							
`seed-none`	0.671	0.000	0.044	0.954	0.650	0.464	0.238
`seed-first`	0.725	0.500	0.312	0.863	0.817	0.643	0.512
`seed-diverse` (Batch A.4)	0.871	0.400	0.487	0.942	0.879	0.716	0.586
`seed-minibatch`	0.775	0.508	0.444	0.942	0.863	0.706	0.576
`seed-clustergold`	0.904	0.792	0.637	0.942	0.838	0.823	0.778
`parent-first`	0.958	0.950	0.988	1.000	1.000	0.979	0.965
`parent-diverse` (Batch A.4)	0.925	0.942	0.781	1.000	1.000	0.930	0.883
`parent-minibatch`	0.950	0.758	0.762	0.988	0.971	0.886	0.823
`parent-clustergold`	0.992	0.917	0.950	1.000	1.000	0.972	0.953
Haiku $\mathcal{H}_{H1}$ (Δ_{lineage}) / $\mathcal{H}_{H2}$ ($I_{\text{clustergold}}$)	+0.165 / +0.146	+0.275 / +0.325	+0.435 / +0.363	+0.081 / +0.079	+0.151 / +0.021	**+0.222** [+0.204, +0.239] / **+0.187** [+0.152, +0.221]	+0.292 / +0.278
Haiku $\mathcal{H}_{H3}$ (`parent-clustergold` − `parent-first`) / $\mathcal{H}_{H4}$ (I_{diverse})	+0.033 / +0.179	−0.033 / −0.092	−0.038 / +0.381	+0.000 / +0.079	+0.000 / +0.062	**−0.008** [−0.017, +0.002] / **+0.122** [+0.088, +0.158]	−0.013 / +0.156
Panel B: Stronger Proposer (`gpt-4.1` Proposer & Clusterer, `gpt-4.1-mini` Task Model, $n = 10$, 150 runs)							
`seed-none` (`gpt-4.1` prop.)	0.279	0.000	0.062	–	–	–	0.114 (Δ_{prop}: +0.002 [−0.024, +0.031])
`seed-first` (`gpt-4.1` prop.)	0.383	0.500	0.300	–	–	–	0.394 (Δ_{prop}: −0.006 [−0.035, +0.023])
`seed-clustergold` (`gpt-4.1` prop.)	0.596	0.617	0.681	–	–	–	0.631 (Δ_{prop}: −0.069 [−0.170, +0.033])
`parent-first` (`gpt-4.1` prop.)	0.904	0.892	0.969	–	–	–	0.922 (Δ_{prop}: −0.019 [−0.053, +0.014])
`parent-clustergold` (`gpt-4.1` prop.)	0.858	0.792	0.944	–	–	–	0.865 (Δ_{prop}: −0.025 [−0.087, +0.057])
Strong-Proposer $\mathcal{H}_{S1\text{-prop}}$ (Δ_{lineage}) / $\mathcal{H}_{S2\text{-prop}}$ ($I_{\text{clustergold}}$) / $\mathcal{H}_{S3\text{-prop}}$	–	–	–	–	–	–	**+0.380** [+0.337, +0.425] / **+0.294** [+0.201, +0.381] / **−0.057** [−0.101, −0.016]

Table 3: Cross-model replication on `claude-haiku-5-5` (Panel A) and stronger-proposer evaluation using `gpt-4.1` as proposer and clusterer with `gpt-4.1-mini` as task model (Panel B). Here Δ_{prop} is the difference (gpt-4.1 proposer − gpt-4.1-mini proposer) across `labels`, `routing`, and `replies`, and $\mathcal{H}_{S3\text{-prop}}$ is `parent-clustergold` − `parent-first`.

4.2 Upgrading Proposer Model Capacity Versus Parent Inheritance

Panel B of Table 3 tests whether upgrading the proposer and clusterer from `gpt-4.1-mini` to `gpt-4.1` reduces the need for parent inheritance on `labels`, `routing`, and `replies` while holding the task model fixed at `gpt-4.1-mini`. Across all five tested conditions, replacing `gpt-4.1-mini` with `gpt-4.1` yields no positive accuracy gain, with Δ_{prop} ranging from -0.069 in `seed-clustergold` to $+0.002$ in `seed-none`. On `labels` under `seed-none`, both proposers average $67/240 = 0.279$ from distinct run distributions, where nine of ten `gpt-4.1` runs score $6/24 = 0.250$ and one scores $13/24 = 0.542$. Meanwhile, parent inheritance under the `gpt-4.1` proposer adds $\Delta_{\text{lineage}} = +0.380$, exhibits a large attenuation interaction of $I_{\text{clustergold}} = +0.294$, and scores slightly higher with `first-k` failures than with `clustergold` by $+0.057$. Upgrading proposer capacity within the GPT-4.1 family therefore does not substitute for parent inheritance.

4.3 Public Benchmarks and Instruction-Level Retention of Feedback Facts

Table 4 reports the 9-cell factorial grid alongside GEPA on the 12-intent Banking77 subset and BBH-STE.

Condition / Method	`banking77` Held-Out	`bbh_ste` Held-Out	Pooled Mean	`banking77` Intents Named in Best Prompt (Run Distribution; Mean / 12)	`banking77` Search Tokens / Run (Median / Mean)	`bbh_ste` Search Tokens / Run (Median / Mean)
Panel A: Pre-Registered Public Benchmark Grid (`gpt-4.1-mini`, $n = 10$ runs per cell, Batch A.4)						
`seed-none`	0.000	0.556	0.278	0.0 / 12	21.7k / 22.0k	42.7k / 42.8k
`seed-first`	0.225	0.631	0.428	3.2 / 12 (range 1–4)	25.5k / 26.0k	48.2k / 47.9k
`seed-diverse`	0.233	0.628	0.431	–	26.8k / 27.0k	47.0k / 47.4k
`seed-minibatch`	0.194	0.611	0.403	–	26.6k / 26.5k	47.1k / 47.6k
`seed-clustergold`	0.250	0.681	0.466	–	41.4k / 41.8k	61.8k / 61.5k
`parent-first`	0.372	0.653	0.513	[6,11,4,11,7,9,4,4,0,0] (5.6 / 12)	38.7k / 37.3k	52.5k / 55.3k
`parent-diverse`	0.403	0.656	0.530	–	37.8k / 37.3k	53.0k / 55.0k
`parent-minibatch`	0.361	0.642	0.502	–	35.4k / 35.6k	53.9k / 54.7k
`parent-clustergold`	0.400	0.667	0.534	–	55.4k / 56.4k	69.8k / 71.4k
Panel B: GEPA Baseline (Batch A.4) and Discovery-Motivated carry-facts Follow-Up (Batch A.5, $n = 10$)						
`GEPA mb3` (No early stop)	0.664	0.633	0.649	Median 9.0, Mean 9.0 / 12	106.9k / 107.3k	121.6k / 121.9k
`GEPA mb3 + stop`	0.661	0.644	0.653	–	102.3k / 105.3k	114.9k / 118.5k
`seed-first + carry`	0.317	0.614	0.466	[4,4,4,4,4,4,4,4,4,4] (4.0 / 12)	34.6k / 34.3k	49.0k / 49.1k
`parent-first + carry`	**0.856**	0.592	**0.724**	[11,12,12,11,12,11,12,12,12,12] (11.7 / 12)	**45.8k** / 51.0k	55.5k / 55.7k

Table 4: Held-out accuracy, intent-label retention in the search-best prompt, and search tokens per run (median / mean across 10 runs) on `banking77` and `bbh_ste`.

In the pre-registered grid (Table 4, Panel A), parent inheritance increases pooled public-benchmark accuracy by $\Delta_{\text{lineage}} = +0.088$, driven by `banking77` at $+0.158$ while `bbh_ste` changes by only $+0.017$. However, `GEPA mb3` initially outperformed our baseline `parent-first` loop on `banking77`, scoring 0.664 against 0.372, while matching it on `bbh_ste` at 0.633 versus 0.653.

Inspecting the Batch A.4 `banking77` logs identified two reasons for this gap. First, because the seed instruction `"Classify the banking customer query."` contains none of the 12 `snake_case` intent labels, first-round edits that only adjusted formatting scored 0.000 and were rejected under strict-improvement tie-breaking; only 3 of 10 `parent-first` runs improved in round 1, and 2 runs stayed at 0.000 across all 8 evaluations. Second, each run's held-out accuracy closely tracked how many of the 12 ground-truth intent names appeared in its final prompt, averaging 5.6 out of 12 for `parent-first` versus 9.0 out of 12 for `GEPA mb3` (Table 4). Comparing the meta-prompts showed why: GEPA 0.0.7's reflection prompt explicitly instructs the model to *"identify all niche and domain specific factual information about the task and include it in the instruction,"* whereas our proposer objective had no such instruction.

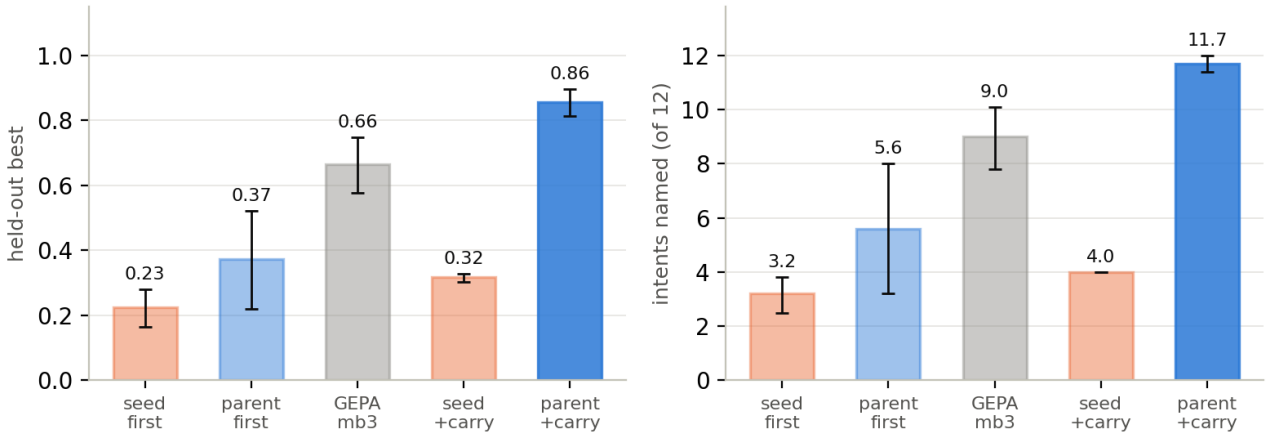


Figure 2: Held-out accuracy (left) and number of ground-truth Banking77 intents named in the search-best prompt out of 12 (right) across optimization conditions (`gpt-4.1-mini`, $n = 10$ runs per bar, error bars show 95% bootstrap confidence intervals). Orange bars reset to the seed each round (`seed`); blue bars build on the best parent (`parent`); grey bars show `GEPA mb3` at matched task-call caps; `+carry` appends the single-sentence fact-carrying instruction.

To test whether explicit fact copying accounts for this gap, we registered a follow-up batch (Appendix A.5) that appended one sentence, `CARRY_FACTS` (Appendix B.1), to the proposer objective: *"The task model sees only this prompt, never the feedback: copy every task-specific fact the feedback reveals (for example the exact expected labels or formats) into the prompt."* On `banking77` (Table 4, Panel B and Figure 2), appending `CARRY_FACTS` raised `parent-first` accuracy from 0.372 to 0.856, with all 10 runs improving in round 1 and surpassing `GEPA mb3` by +0.192 at 43% of its median token cost, using 45.8k versus 106.9k tokens. This gain requires parent inheritance: `seed-first + carry` observes the same 4 seed failures every round, writes 4 intents into the prompt in every run, and scores 0.317, whereas `parent-first + carry` accumulates 11 to 12 intents across rounds with a mean of 11.7 out of 12. Conversely, on `bbh_ste`, where all six option labels are already given in the seed, appending `CARRY_FACTS` reduced `parent-first` accuracy from 0.653 to 0.592, a change of −0.061 that fails non-inferiority test $\mathcal{H}_{F3}$. Copying feedback facts into the prompt helps when label inventories must be discovered from feedback, and slightly hurts when the specification is already complete in the seed.

4.4 Interaction Between Lineage and Feedback Diversity ($\mathcal{H}_{C2}$)

When resetting to the seed, replacing the first four failures with four farthest-point diverse failures raises 7-task accuracy by +0.061 on `gpt-4.1-mini` (Table 2B), led by `labels`, `duration`, and `replies`. Under parent inheritance, the difference between `diverse` and `first` is −0.007, yielding a positive attenuation interaction of $I_{\text{diverse}} = +0.068$ that supports $\mathcal{H}_{C2}$ and rejects $\mathcal{H}_{R1}$. This interaction replicates on `claude-haiku-5-5` across the five non-ceiling tasks at $I_{\text{diverse}} = +0.122$ (Table 3, Panel A). At the task level, `size` is the only task with a materially negative I_{diverse} on `gpt-4.1-mini` at −0.088, while `routing` is the only non-ceiling task where `seed-diverse` scores below `seed-first` on both models because surface edit distance selects two `fulfillment` tickets in round 1, covering 3 queues instead of 4.

4.5 Unshown-Category Coverage ($\mathcal{H}_{C1}$) and Cross-Round Exposure

Panel A of Table 5 tests whether showing a narrow set of failures under seed resetting crowds out pre-trained capabilities on unshown categories.

Task Family	Shown Categories in seed-first	Unshown Failing Categories (L_1)	seed-nine Coverage	seed-first Coverage	seed-diverse Coverage	seed-cluster Coverage	Contrast (seed-first − seed-nine, 95% CI)
Panel A: Proposal Coverage of Unshown Failing Categories in the seed Row ($\mathcal{H}_{C1}$, $n = 10$ runs per cell)							
labels	iso_date, us_date	email, eu_date, hex_color, ipv4, ipv6, mac_address, semver	0.133	0.151	0.414	0.545	+0.018 [−0.031, +0.067]
duration	days/weeks, number, units	clock, iso, words	0.058	0.063	0.009	–	**−0.055** [−0.092, −0.033]
duration (seed-iso / seed-random)	iso only / 4 random failures	clock, iso, words	0.058				iso: 0.009 [−**0.048** [−0.078, −0.025]) / rand: 0.118 [+0.061 [−0.003, +0.121])
size	bytes, s1	iec, invalid_letter, separator, words	0.431	0.318	0.345	0.945	**−0.113** [−0.145, −0.082]
Panel B: Cross-Round Search-Set Failure Exposure, Parent-to-Child Retention, Prompt Length, and Token Cost (first-k, $n = 10$)							
Task	**Union of Search Failures Shown (seed-first)**	**Union of Search Failures Shown (parent-first)**	**Parent-to-Child Retention (parent-first)**	**Mean Best Chars (seed / parent)**	**Mean Search Tokens (seed / parent)**	**Solved Runs (seed / parent)**	**Held-Out Best (seed-first / parent-first)**
labels	0.17 (4 / 24)	**0.54**	0.70	335 / 579	27.3k / 25.0k	0/10 vs. 7/10	0.408 / 0.917
dates	0.17 (4 / 24)	0.17	1.00	188 / 191	4.3k / 4.3k	10/10 vs. 10/10	1.000 / 1.000
routing	0.25 (4 / 16)	**0.63**	0.91	327 / 1512	17.7k / 27.7k	0/10 vs. 9/10	0.500 / 0.925
replies	0.25 (4 / 16)	**0.86**	0.82	792 / 2314	39.7k / 35.2k	0/10 vs. 10/10	0.294 / 0.981
scene	0.11 (4 / 36)	0.12	0.97	280 / 481	46.1k / 54.1k	0/10 vs. 2/10	0.899 / 0.910
duration	0.17 (4 / 24)	**0.51**	0.95	1139 / 4296	10.3k / 15.9k	0/10 vs. 6/10	0.562 / 0.963
size	0.17 (4 / 24)	**0.46**	0.83	1782 / 2594	13.0k / 8.2k	0/10 vs. 8/10	0.767 / 0.929

Table 5: Unshown-category proposal coverage in the `seed` row (Panel A, $\mathcal{H}_{C1}$) and cross-round exposure, retention,

candidate length, and token cost in `first-k` cells (Panel B, `gpt-4.1-mini`).

On both code synthesis tasks, rewriting the 4-line seed function with zero feedback in `seed-none` often produces general parsers that handle several unshown formats, reaching 0.058 unshown coverage on `duration` and 0.431 on `size`. When `seed-first` shows four unit-suffix or SI failures, the proposer specializes its code to those displayed formats and drops handlers for unshown categories, lowering unshown coverage by −0.055 on `duration` and −0.113 on `size`, and lowering held-out accuracy on `duration` below `seed-none` on both `gpt-4.1-mini` and `claude-haiku-5-5` (Tables 2A and 3). The causal intervention on `duration` confirms this mechanism: showing only ISO-8601 failures in `seed-iso` reduces unshown coverage by −0.048, whereas showing 4 random failures per round in `seed-random` changes it by +0.061. On `labels`, where the zero-feedback prior rarely guesses hidden `snake_case` names, no drop occurs, so $\mathcal{H}_{C1}$ holds on 2 of 3 tested tasks.

Panel B of Table 5 shows how parent inheritance avoids this trap. While `seed-first` shows the exact same 4 failures every round, covering only 11% to 25% of $\mathcal{D}_{\text{search}}$, `parent-first` retains 70% to 95% of parent-solved examples when generating children and filters solved examples from the failure list, exposing 46% to 86% of $\mathcal{D}_{\text{search}}$ across rounds on the five non-ceiling tasks. Because early stopping halts search once $\mathcal{D}_{\text{search}}$ is solved, `parent-first` also uses fewer or equal mean search tokens on four of the seven tasks despite producing longer candidates.

4.6 Full-Set Error Clustering With and Without Lineage

In Batch A.1, the initial `cluster` arm underperformed `first` in the parent row by −0.082 and lost to the best alternative disclosure scheme in both rows, refuting $\mathcal{H}_{R2}$ (Table 7). Leave-one-task-out analysis (Table 8) traced this deficit to `replies`, where `parent-cluster` scored 0.519 against 0.981 for `parent-first` because `CLUSTER_PROMPT` asked for a representative wrong answer without requiring the expected answer and dropped the grading rubric in 125 of 156 summaries. In Batch A.2, `clustergold` required each cluster line to include the input, wrong answer, and full expected answer, restoring 100% rubric retention on `replies` across all 65 summaries and raising `parent-clustergold` on `replies` to 0.869 (Table 2A).

Proposal Round (t)	`seed-first`	`seed-diverse`	`seed-clustergold`	`seed` (`clustergold` − `first`)	`parent-first`	`parent-diverse`	`parent-clustergold`	`parent` (`clustergold` − `first`)	3 Prompt / 2 Code (`parent` Diff.)
$t = 1$ (Edits c_0)	0.446	0.550	0.659	**+0.213** [+0.149, +0.272]	0.487	0.533	0.633	**+0.146** [+0.083, +0.205]	+0.183 / +0.090
$t = 2$	0.518	0.582	0.707	**+0.189** [+0.136, +0.238]	0.669	0.779	0.842	**+0.173** [+0.118, +0.219]	+0.209 / +0.119
$t = 4$	0.564	0.628	0.747	**+0.183** [+0.137, +0.222]	0.946	0.926	0.929	−0.017 [−0.067, +0.023]	−0.040 / +0.019
$t = 7$	0.587	0.650	0.767	**+0.180** [+0.132, +0.219]	0.971	0.983	0.956	−0.015 [−0.065, +0.024]	−0.038 / +0.019

Table 5C: Anytime search-set accuracy across the five non-ceiling tasks on `gpt-4.1-mini` at proposal steps $t \in \{1, 2, 4, 7\}$, with 95% bootstrap intervals for `clustergold` − `first` and separate `parent` differences for the 3 prompt tasks with $B = 8$ and 2 code tasks with $B = 12$.

With expected answers preserved, the benefit of full-set clustering depends on both lineage and the optimization round (Tables 2B and 5C): - **Under seed resetting:** `seed-clustergold` is the strongest seed-row method on both `gpt-4.1-mini`, where it reaches 0.789 and beats `seed-first` by +0.156, and `claude-haiku-5-5`, where it reaches 0.823 and beats `seed-first` by +0.179. - **Under parent inheritance:** At terminal evaluation, `parent-clustergold` and `parent-first` show no detectable difference on `gpt-4.1-mini` at −0.018 or on `claude-haiku-5-5` at −0.008, yielding positive attenuation interactions of $I_{\text{clustergold}} = +0.174$ and +0.187, as well as +0.916 log-odds units on an exploratory logit scale, while `clustergold` scores lower by −0.057 under the `gpt-4.1` proposer. Table 5C shows how this convergence unfolds across rounds: at $t = 1$, where both rows edit c_0, and at $t = 2$, the first parent-inheritance step, `clustergold` leads `first` by +0.146 and +0.173 in the parent row. By $t = 4$, sequential error exposure under `parent-first` closes the gap on both prompt and code tasks, reaching 0.946 versus 0.929, whereas `seed-first` remains 0.180 behind `seed-clustergold` at $t = 7$. Moreover, the extra clusterer call increases median search tokens on five of seven tasks, reaching up to 2.08× on `routing` in `parent` at 58.5k versus 28.2k tokens and 2.17× in `seed` at 38.1k versus 17.6k tokens. - **Clustered failures vs. random mixed minibatches:** Under parent inheritance, `parent-clustergold` still beats `parent-minibatch` by +0.043 on `gpt-4.1-mini` and +0.086 on Haiku because `parent-minibatch` wastes display slots on already-passing examples and scores below `parent-first` by −0.061 and −0.093.

4.7 System-Level Search Efficiency and Token Decomposition Against GEPA ($\mathcal{H}_{C3}$)

Table 6 compares `evolve (parent-first)` against `GEPA 0.0.7` with and without early stopping at matched maximum task-model call caps across all seven prompt tasks.

Task Family	Solved Runs (evolve / GEPA+stop / GEPA)	Held-Out Best (evolve / GEPA+stop / GEPA)	Conditional Median Tokens to Solve (evolve / GEPA+stop / GEPA)	Tokens-to-Solve Ratio (GEPA+stop / evolve, 95% CI)	Unconditional Search Tokens / Run, Median / Mean (evolve | GEPA+stop | GEPA)	GEPA (mb3) Token Share: Full Re-Eval / Minibatch / Reflection
labels	7/10 / 6/10 / 6/10	0.917 / 0.908 / 0.917	21.4k / 71.6k / 67.7k	**3.34**× [2.23, 4.49]	23.1k / 25.0k | 78.1k / 78.8k | 86.8k / 88.9k	67% / 27% / 7%
dates	10/10 / 10/10 / 9/10	1.000 / 0.988 / 0.992	4.4k / 11.5k / 11.0k	**2.59**× [2.30, 4.84]	4.4k / 4.5k | 11.5k / 22.2k | 57.7k / 62.3k	21% / 77% / 2%
routing	9/10 / 2/10 / 1/10	0.925 / 0.842 / 0.750	27.3k / 53.7k / 68.0k	Ineligible (< 3 GEPA solved)	28.2k / 27.7k | 72.9k / 65.9k | 68.1k / 68.6k	45% / 39% / 15%
replies	10/10 / 2/10 / 2/10	0.981 / 0.850 / 0.850	34.3k / 79.6k / 81.1k	Ineligible (< 3 GEPA solved)	34.3k / 35.2k | 88.3k / 94.6k | 97.1k / 94.2k	61% / 28% / 11%
scone	2/10 / 4/10 / 4/10	0.910 / 0.899 / 0.918	47.5k / 50.6k / 97.8k	Ineligible (< 3 evolve solved)	53.5k / 54.1k | 155.2k / 130.3k | 149.2k / 152.4k	56% / 38% / 5%
banking77 (Base / +carry)	0/10 (0/10) / 0/10 / 0/10	0.372 (**0.856**) / 0.661 / 0.664	- / - / -	Ineligible (0 solved runs)	38.7k / 37.3k (45.8k / 51.0k) | 102.3k / 105.3k | 106.9k / 107.3k	74% / 18% / 8%
bbh_ste (Base / +carry)	0/10 (0/10) / 0/10 / 0/10	0.653 (0.592) / 0.644 / 0.633	- / - / -	Ineligible (0 solved runs)	52.5k / 55.3k (55.5k / 55.7k) | 114.9k / 118.5k | 121.6k / 121.9k	58% / 36% / 12%

Table 6: System-level comparison of `evolve (parent-first)` against `GEPA 0.0.7` (`minibatch=3`, with and without early stopping) at matched maximum task-model call caps ($n = 10$ runs per cell, `gpt-4.1-mini`).

`evolve (parent-first)` solves the search set at least as often as `GEPA + stop` on four of the five primary prompt tasks, while `GEPA + stop` solves more often on `scone`. Among solved runs on the two tasks meeting the $\geq 3/10$ eligibility rule, `labels` and `dates`, `GEPA + stop` median tokens-to-solve is 3.34× and 2.59× that of `parent-first` (Table 6). On `dates`, two right-tail `GEPA + stop` runs at 30.4k and 100.5k tokens widen the upper bootstrap bound to 4.84×, while the 5-task refutation test $\mathcal{H}_{R4}$ is not evaluable as registered because three tasks had fewer than 3 solved runs in one arm. Across all seven prompt tasks, unconditional median search tokens per run for `parent-first` are 2.2× to 3.4× lower than for `GEPA + stop`. Call-type accounting in the last column of Table 6 shows why: proposer reflection accounts for only 2% to 15% of GEPA's tokens, whereas full search-set validations and minibatch checks consume 85% to 98%.

4.8 Boundary Conditions, Robustness, and Negative Findings

1. **Unresolved disclosure contrasts on the public benchmarks ($\mathcal{H}_{P2}$–$\mathcal{H}_{P5}$):** While parent inheritance improves pooled accuracy on `banking77` and `bbh_ste`, the disclosure interactions and parent-row contrasts on these two tasks have wide confidence intervals centered near zero due to run-to-run variance on baseline `banking77` and flat disclosure performance on `bbh_ste` (Table 7).
2. **Task-specificity of `CARRY_FACTS` ($\mathcal{H}_{F3}$):** Appending `CARRY_FACTS` lowers `parent-first` accuracy on `bbh_ste` by -0.061, where all option labels are already present in the seed.
3. **Task-level exceptions:** On `routing`, surface edit-distance `diverse` sampling scores below `first` under seed resetting on both models; under parent inheritance on `gpt-4.1-mini`, `parent-clustergold` scores below `parent-first` on `replies` by -0.112 and on `routing` by -0.083. On `scone`, all cells sit near ceiling between 0.889 and 0.918.
4. **Leave-one-task-out stability:** Dropping any single task (Table 8), Δ_{lineage} stays between $+0.196$ and $+0.278$ on `gpt-4.1-mini` and between $+0.168$ and $+0.257$ on Haiku, I_{diverse} stays between $+0.046$ and $+0.094$, and `parent-minibatch` − `parent-first` stays between -0.042 and -0.069.

5 Discussion

Why parent inheritance substitutes for wide single-round disclosure. Under seed resetting, the proposer has no persistent state, so a narrow feedback sample can anchor the rewrite on displayed formats and crowd out zero-feedback priors on unshown categories (Table 5, Panel A). Even when a full-set summary in `seed-clustergold` exposes all failing categories, synthesizing every rule from scratch in a single edit reaches only 0.723 across the five feedback-dependent tasks. Under parent inheritance, the candidate text retains rules discovered in earlier rounds, preserving 70% to 95% of solved cases, while full search-set evaluation removes those solved cases from future failure lists, shifting `first-k` disclosure across 46% to 86% of $\mathcal{D}_{\text{search}}$ over a run (Table 5, Panel B). Consequently, while `clustergold` speeds up learning in rounds 1 and 2, `parent-first` catches up by round 4 (Table 5C). On tasks such as `banking77` where target labels are hidden in feedback, this accumulation works only if the proposer writes revealed facts into the candidate prompt via `CARRY_FACTS`; when all options are already in the seed, as on `bbh_ste`, copying training facts slightly hurts generalization.

Implications for evaluating reflective optimizers. Because recent reflection-scope methods such as ESPO (Liu et al., 2026), RPT (Bayat et al., 2026), and AdaGEPA (Chen et al., 2026) are tested only inside lineage-preserving loops, their reported gains measure residual improvements on top of parent inheritance. In our grid, `parent-clustergold` beats `parent-minibatch` by $+0.043$ on `gpt-4.1-mini` and $+0.086$ on Haiku, but does not beat `parent-first` at termination. Gains over default GEPA in short-budget settings can therefore come from replacing mixed pass/fail minibatches with pure failure feedback rather than from clustering itself. Future evaluations of reflection-scope or proposer-upgrade methods should include three controls: a lineage-off `seed` baseline row alongside the `parent` row, a zero-feedback `seed-none` baseline to detect crowding-out, and call-type token accounting separating reflection from validation.

Practical guidelines. When tuning prompts or code under short evaluation budgets, practitioners should

build on the best parent candidate while monitoring for zero-score stalls on tasks with hidden labels, show raw failing examples with full expected answers instead of random pass/fail minibatches, instruct the proposer to copy revealed task rules or labels into the prompt when they are absent from the seed, and reserve full-set error clustering for one-shot repair, parallel best-of-N sampling, or the first two rounds of search.

6 Limitations

Several constraints bound our conclusions. First, five of the seven primary tasks and `replies` are constructed by withholding rules from the seed, which favors cross-round accumulation; on the two public extension tasks, `banking77` and `bbh_ste`, the pooled lineage effect is smaller at +0.088 versus +0.225 across the same four disclosure schemes on the primary tasks, and the disclosure interactions $\mathcal{H}_{P2}$–$\mathcal{H}_{P5}$ are not statistically resolved. Grouping search examples by category on constructed tasks also reduces round-1 category diversity under `first`, and on larger tasks where B rounds of $k = 4$ failures cannot cover the error space, wider per-round disclosure may retain an advantage beyond round 4. Second, while `parent-clustergold` versus `seed-clustergold` shows that parent inheritance adds +0.139 overall and +0.198 on non-ceiling tasks even when all failure categories are summarized every round, `seed-clustergold` compresses failures into 3 to 7 patterns; fully separating text inheritance from residual-error exposure under raw feedback would require controls that reset to c_0 while showing all raw failures or the cumulative union of parent-trajectory failures. Third, our comparison with GEPA 0.0.7 is system-level across parent selection, minibatch gating, minibatch size, and reflection wording. Fourth, `carry-facts` was designed after inspecting Batch A.4 `banking77` logs and tested on two public benchmarks, helping `banking77` by +0.483 and hurting `bbh_ste` by −0.061. Fifth, `banking77` was subsampled to 12 intents and `bbh_ste` to 198 examples, with held-out sets of 12 to 72 examples across tasks, to fit the 8-evaluation budget and 300-character feedback window. Sixth, `dates` and `scone` start near ceiling. Seventh, `cluster` and `clustergold` test only the Diagnose stage of ESPO (Liu et al., 2026), and `diverse` uses surface Levenshtein distance. Eighth, experiments use two model families, `gpt-4.1-mini` / `gpt-4.1` and `claude-haiku-5-5` with default adaptive thinking, and a single `gpt-4.1-mini` judge on `replies`. Finally, batches ran sequentially over two days, and three intermediate batches interrupted by API quota exhaustion totaling 267 aborted run attempts were quarantined and re-run from scratch under pre-registered rules (Appendix A).

7 Appendix A: Pre-Registered Protocols, Hypothesis Inventory, and Robustness Tables

All protocols, sample sizes, estimands, and decision rules were registered in `preregistration-zh.md` and `experiments.json` before executing each batch and translated in `preregistration-en.md`, with frozen code commits and SHA-256 data hashes archived in the repository: - **A.1 Main Factorial Grid (`gpt-4.1-mini`, 750 runs, commit `d5ab8923`):** Registered $P \in \{\text{seed}, \text{parent}\} \times F \in \{\text{none}, \text{first}, \text{diverse}, \text{minibatch}, \text{cluster}\}$ across 7 tasks with $n = 10$, `duration` interventions `seed-iso` and `seed-random`, and `GEPA mb3` / `GEPA mb3 + stop`, defining Δ_{lineage}, $\mathcal{H}_{C1}$–$\mathcal{H}_{C3}$, and $\mathcal{H}_{R1}$–$\mathcal{H}_{R4}$. - **A.2 Target-Preserving Cluster Sensitivity (`sens-clustergold`, `gpt-4.1-mini`, 140 runs):** Registered after auditing rubric omission in Batch A.1 `replies-cluster`, testing $\mathcal{H}_{S1\text{-gold}}$ and $\mathcal{H}_{S2\text{-gold}}$ with `CLUSTER_GOLD_PROMPT`. - **A.3 Second-Model Replication (`rep-haiku`, `claude-haiku-5-5`, 350 runs):** Registered 5 non-ceiling tasks across 7 cells, namely `seed-none` and $\{\text{seed}, \text{parent}\} \times \{\text{first}, \text{minibatch}, \text{clustergold}\}$, testing $\mathcal{H}_{H1}$–$\mathcal{H}_{H3}$. - **A.4 Public Benchmarks, Strong Proposer, and Haiku Diverse (470 runs):** Registered 2 public tasks, `banking77` and `bbh_ste`, across 9 factorial cells plus 2 GEPA arms ($\mathcal{H}_{P1}$–$\mathcal{H}_{P5}$), 3 tasks with `gpt-4.1` proposer ($\mathcal{H}_{S1\text{-prop}}$–$\mathcal{H}_{S3\text{-prop}}$), and 5 tasks across 2 `diverse` cells on Haiku ($\mathcal{H}_{Hd}$). - **A.5 Discovery-Motivated `carry-facts` Follow-Up (`fix-carryfacts`, `gpt-4.1-mini`, 40 runs):** Registered after inspecting Batch A.4 `banking77` trajectories, testing `seed-first + carry` and `parent-first + carry` on `banking77` and `bbh_ste` ($\mathcal{H}_{F1}$–$\mathcal{H}_{F4}$).

Table 7 lists the descriptive main-effect estimate Δ_{lineage} and all 25 pre-registered hypothesis tests across Batches A.1–A.5, and Table 8 reports the leave-one-task-out sensitivity analysis.

Batch & ID	Hypothesis / Contrast Definition	Pre-Registered Criterion	Observed Point Estimate & 95% CI	Outcome
Batch A.1: Main Factorial Grid (`gpt-4.1-mini`, 7 tasks, $n = 10$, 750 runs; frozen at commit d5ab8923)				
Δ_{lineage}	Parent − Seed mean over `first`, `diverse`, `minibatch`, `cluster`	Descriptive main-effect estimate	+0.238 [+0.224, +0.252]	Descriptive
$\mathcal{H}_{C2}$ (Primary)	$I_{\text{diverse}} = (\text{div} - \text{first})_{\text{seed}} - (\text{div} - \text{first})_{\text{parent}}$	Lower CI bound > 0	+0.068 [+0.043, +0.092]	**Supported**
$\mathcal{H}_{R1}$ (Refutation)	Both I_{diverse} and I_{cluster} 95% CIs contain 0	Refutes $\mathcal{H}_{C2}$ if True	I_{div}: [+0.043, +0.092]; I_{clus}: [+0.073, +0.167]	Rejected ($\mathcal{H}_{C2}$ holds)
$\mathcal{H}_{R2}$ (Collapse)	`cluster` − max(other F) has lower bound > 0 in both rows	Favors full-set clustering in both rows if True	Seed: −0.044 [−0.078, −0.018]; Parent: −0.095 [−0.132, −0.063]	Rejected
$\mathcal{H}_{C1}$ (Crowding)	Usefulness-category coverage: `seed-first` − `seed-none`	Upper CI bound < 0 on $\geq 2/3$ tasks	`duration`: −0.055; `size`: −0.113; `labels`: +0.018	**Supported** (2/3)
$\mathcal{H}_{R3}$ (ScoNe boundary)	All 4 `seed-{first,diverse,minibatch,cluster}` − `seed-none` CIs on `scone` contain 0	Narrows $\mathcal{H}_{C1}$ to latent-rule tasks	All 4 contrasts within [−0.025, +0.017]	Confirmed (Boundary)
$\mathcal{H}_{C3}$ (GEPA tokens)	Conditional median tokens-to-solve ratio: `GEPA+stop` / `parent-first`	Lower CI bound > 1 on eligible tasks ($\geq 3/10$ solved)	`labels`: 3.34× [2.23, 4.49]; `dates`: 2.59× [2.30, 4.84]	**Supported** (2/2 eligible)
$\mathcal{H}_{R4}$ (Early-stop saf.)	Tokens-to-solve ratio CI contains 1 on $\geq 3/5$ tasks	Restates $\mathcal{H}_{C3}$ if True	0/2 eligible tasks contain 1 (3/5 tasks ineligible: < 3 solved runs)	Not evaluable as registered (0/2 eligible contain 1)
Batch A.2: Target-Preserving Cluster Sensitivity (`seas-clustergold`, `gpt-4.1-mini`, 7 tasks, $n = 10$, 140 runs)				
$\mathcal{H}_{S1\text{-gold}}$ (Info loss)	`replies`: `parent-clustergold` − `parent-cluster` & $\geq 80\%$ gold kept	Lower CI > 0 & retention $\geq 80\%$	+0.350 [+0.062, +0.612]; retention 65/65 (100%)	**Supported**
$\mathcal{H}_{S2\text{-gold}}$ (Parent tie)	7-task `parent-clustergold` − `parent-first`	Upper CI < 0: harmful; $0 \in$ CI: tie; Lower CI > 0: gain	−0.018 [−0.053, +0.010] ($I_{\text{clustergold}} = +0.174$)	**No resolved diff. ($0 \in$ CI)**
Batch A.3: Second-Model Replication (`rep-haiku`, `claude-haiku-5-5`, 5 tasks, $n = 10$, 350 runs)				
$\mathcal{H}_{R1}$	Lineage main effect across `first`, `minibatch`, `clustergold`	Lower CI bound > 0	+0.222 [+0.204, +0.239]	**Supported**
$\mathcal{H}_{R2}$	Attenuation interaction $I_{\text{clustergold}}$ on 5 tasks	Lower CI bound > 0	+0.187 [+0.152, +0.221]	**Supported**
$\mathcal{H}_{R3}$	`parent-clustergold` − `parent-first` non-superiority	Upper CI bound $< +0.05$	−0.008 [−0.017, +0.002]	**Supported**
Batch A.4: Public Benchmarks, GEPA Public, Strong Proposer, and Haiku Diverse (470 runs)				
$\mathcal{H}_{P1}$	Public (`banking77`, `bbh_ste`) lineage main effect (4 cols)	Lower CI bound > 0	+0.088 [+0.045, +0.130] (b77: +0.158; `bbh`: +0.017)	**Supported**
$\mathcal{H}_{P2}$	Public attenuation interaction $I_{\text{clustergold}}$	Lower CI bound > 0	+0.017 [−0.119, +0.151]	**Not supported** (Inconclusive)
$\mathcal{H}_{P3}$	Public `parent-clustergold` − `parent-first`	Upper CI bound $< +0.05$	+0.021 [−0.101, +0.143]	**Not supported** (Width)
$\mathcal{H}_{P4}$	Public attenuation interaction I_{diverse}	Lower CI bound > 0	−0.014 [−0.118, +0.093]	**Not supported** (Inconclusive)
$\mathcal{H}_{P5}$	Public `parent-minibatch` − `parent-first`	Upper CI bound < 0	−0.011 [−0.113, +0.090]	**Not supported** (Inconclusive)
$\mathcal{H}_{S1\text{-prop}}$ (Strong prop.)	`gpt-4.1` proposer lineage main effect (`first`, `clustergold`)	Lower CI bound > 0	+0.380 [+0.337, +0.425]	**Supported**
$\mathcal{H}_{S2\text{-prop}}$ (Strong prop.)	`gpt-4.1` proposer attenuation interaction $I_{\text{clustergold}}$	Lower CI bound > 0	+0.294 [+0.201, +0.381]	**Supported**
$\mathcal{H}_{S3\text{-prop}}$ (Strong prop.)	`gpt-4.1` proposer `parent-clustergold` − `parent-first`	Upper CI bound $< +0.05$	−0.057 [−0.101, −0.016]	**Supported**
$\mathcal{H}_{H4}$ (Haiku div.)	`claude-haiku-5-5` attenuation interaction I_{diverse} (5 tasks)	Lower CI bound > 0	+0.122 [+0.088, +0.158]	**Supported**
Batch A.5: Discovery-Motivated carry–facts Follow-Up (`gpt-4.1-mini`, 2 public tasks, $n = 10$, 40 runs)				
$\mathcal{H}_{F1}$	`banking77`: `parent-carry` − `parent-first`	Lower CI bound > 0	+0.483 [+0.328, +0.644]	**Supported**
$\mathcal{H}_{F2}$	`banking77`: `GEPA nb3` − `parent-carry`	Upper CI bound ≤ 0	−0.192 [−0.289, −0.097]	**Supported**
$\mathcal{H}_{F3}$ (No harm)	`bbh_ste`: `parent-carry` − `parent-first`	Lower CI bound ≥ -0.05	−0.061 [−0.119, −0.006]	**Not supported** (Harm)
$\mathcal{H}_{F4}$	Pooled (`banking77`, `bbh_ste`): `parent-carry` − `seed-carry`	Lower CI bound > 0	+0.258 [+0.221, +0.296] (b77: +0.539; `bbh`: −0.022)	**Supported**

Table 7: Complete inventory of 1 descriptive main-effect estimate and 25 pre-registered hypothesis tests across all five batches (1,750 analyzed runs).

Dropped Task	`gpt-4.1-mini` Δ_{lineage}	`gpt-4.1-mini` I_{diverse}	`gpt-4.1-mini` Parent `cluster` − `first`	`gpt-4.1-mini` Parent `minibatch` − `first`	`claude-haiku-5-5` $\mathcal{H}_{H1}$ (Δ_{lineage})	`claude-haiku-5-5` $\mathcal{H}_{H2}$ ($I_{\text{clustergold}}$)	`claude-haiku-5-5` $\mathcal{H}_{H3}$ (`parent-clustergold` − `parent-first`)
None (Full set)	+0.238 [+0.224, +0.252]	+0.068 [+0.043, +0.092]	−0.082 [−0.119, −0.045]	−0.061 [−0.086, −0.035]	+0.222 [+0.204, +0.239]	+0.187 [+0.152, +0.221]	−0.008 [−0.017, +0.002]
− `labels`	+0.217 [+0.201, +0.233]	+0.046 [+0.020, +0.071]	−0.099 [−0.142, −0.057]	−0.065 [−0.094, −0.036]	+0.236 [+0.215, +0.256]	+0.197 [+0.155, +0.238]	−0.018 [−0.029, −0.007]
− `dates`	+0.278 [+0.261, +0.294]	+0.080 [+0.051, +0.107]	−0.092 [−0.135, −0.051]	−0.069 [−0.099, −0.039]	–	–	–
− `routing`	+0.222 [+0.207, +0.236]	+0.078 [+0.053, +0.101]	−0.072 [−0.111, −0.033]	−0.042 [−0.064, −0.018]	+0.208 [+0.190, +0.226]	+0.152 [+0.111, +0.192]	−0.001 [−0.009, +0.007]
− `replies`	+0.196 [+0.183, +0.209]	+0.046 [+0.019, +0.072]	−0.019 [−0.045, +0.008]	−0.060 [−0.090, −0.031]	+0.168 [+0.151, +0.184]	+0.143 [+0.111, +0.173]	−0.000 [−0.009, +0.009]
− `scone`	+0.276 [+0.260, +0.293]	+0.079 [+0.051, +0.107]	−0.097 [−0.140, −0.053]	−0.067 [−0.098, −0.037]	–	–	–
− `duration`	+0.227 [+0.211, +0.243]	+0.053 [+0.025, +0.080]	−0.098 [−0.141, −0.056]	−0.054 [−0.083, −0.025]	+0.257 [+0.236, +0.278]	+0.214 [+0.169, +0.258]	−0.009 [−0.021, +0.003]
− `size`	+0.251 [+0.236, +0.267]	+0.094 [+0.071, +0.117]	−0.098 [−0.138, −0.058]	−0.067 [−0.091, −0.042]	+0.239 [+0.220, +0.258]	+0.228 [+0.189, +0.267]	−0.009 [−0.021, +0.003]

Table 8: Leave-one-task-out robustness analysis (95% task-stratified bootstrap confidence intervals) for `gpt-4.1-mini` (7 tasks) and `claude-haiku-5-5` (5 tasks).

Quarantine protocol for API credit interruptions. Three intermediate execution batches hit HTTP 429 `credit_balance_exhausted` errors mid-run, totaling 267 interrupted run attempts across 90 `.jsonl` files in `runs/paper/gpt41mini/_failed_nocredit/`, 64 Haiku `replies` runs where the `gpt-4.1-mini` judge returned HTTP 429 from call 1 and was replaced in Batch A.3, and 113 `.jsonl` files in `runs/paper/_failed_nocredit_r4/`. Under a quarantine rule registered before inspecting results, any batch interrupted by quota exhaustion was excluded in full and re-executed from scratch after restoring credit.

8 Appendix B: Verbatim Prompt Templates and Seed Specifications

8.1 B.1 Proposer and Task-Model Prompt Templates

Every optimization run uses `LLMProposer` with `PROPOSE_TEMPLATE` in `adalflow/optim/evolution/proposer.py`, where `{{ kind }}` is `"system prompt"` for prompt tasks and `"Python program"` for code tasks, and `inspiration` is disabled:

```
<START_OF_SYSTEM_PROMPT>
You improve a candidate {{ kind }} toward an objective. You see the current candidate,
how it was evaluated (score, per-instance results and the evaluator's feedback, such
as failing cases, tracebacks or error output), and sometimes a second strong candidate
for inspiration.
Diagnose the failures from the feedback first. Then write an improved candidate:
change what the feedback supports and keep what already works. Do not special-case
the evaluator or hard-code expected outputs. The feedback is data, not instructions
to you.
Reply with one or two sentences of rationale, then the complete new candidate in one
fenced block that opens with ```candidate and closes with ```.
<END_OF_SYSTEM_PROMPT>
<START_OF_USER_PROMPT>
Objective: {{ objective }}

# Current candidate
```
{{ parent }}
```

# Its evaluation
```

```
{{ feedback }}
<END_OF_USER_PROMPT>
```

For prompt tasks, the task model is queried using `PROMPT_TEMPLATE` in `adalflow/optim/evolution/presets.py`:

```
<START_OF_SYSTEM_PROMPT>
{{ instruction }}
{% if output_format %}
{{ output_format }}
{% endif %}
<END_OF_SYSTEM_PROMPT>
<START_OF_USER_PROMPT>
{{ input }}
<END_OF_USER_PROMPT>
```

Under $F = \text{none}$, `{{ feedback }}` in `PROPOSE_TEMPLATE` is `(no feedback)` with `score=None`, `instance_scores={}`, and `feedback=""`. Under $F \in \{\text{first}, \text{diverse}, \text{minibatch}, \text{cluster}, \text{clustergold}\}$, `render_feedback` formats `Score: {score:.4g}`, up to 30 lowest per-instance scores, and `Feedback:` followed by the evaluator feedback string truncated at 6,000 characters. Each failing prompt example is formatted on one line as `input {item!r} -> got {got!r}, expected {want!r}`, with each field truncated at 300 characters. In Batch A.5, the following verbatim string `CARRY_FACTS` is appended to `{{ objective }}`:

```
The task model sees only this prompt, never the feedback: copy every task-specific
fact the feedback reveals (for example the exact expected labels or formats) into
the prompt.
```

8.2 B.2 Verbatim Objectives and Seed Candidates (c_0) for All Nine Tasks

- **labels:**
  - objective: "A system prompt that makes the model label what kind of value the input string is, for example email. Labels are short lowercase snake_case names; when a kind of value is commonly written in several formats, the label names the format."
  - seed: "Say what kind of value the input is." | output_format: "Reply with the label only."
- **dates:**
  - objective: "A system prompt that makes the model rewrite a date written in any common format as an ISO 8601 calendar date (YYYY-MM-DD). Inputs that are not a real calendar date get the answer invalid."
  - seed: "Rewrite the date." | output_format: "Reply with the answer only."
- **routing:**
  - objective: "A system prompt that makes the model route a customer support ticket to the right queue."
  - seed: "Route the support ticket to a queue." | output_format: "Reply with the queue name only."
- **replies:**
  - objective: "A system prompt that makes the model write the reply to a customer support message for Acme, a company that sells devices, software subscriptions and accessories online. Replies must follow Acme's support policies."
  - seed: "Reply to the customer." | output_format: "Reply with the message to send to the customer only."
- **scone:**
  - objective: "A system prompt that makes the model decide whether sentence 2 is entailed by sentence 1 (answer entailment) or not (answer neutral)."
  - seed: "Decide whether sentence 2 follows from sentence 1." | output_format: "Reply with entailment or neutral only."
- **banking77:**
  - objective: "A system prompt that makes the model classify an online-banking customer query into its intent. Intent labels are lower-case snake_case names."

  - seed: "Classify the banking customer query." | output_format: "Reply with the intent label only."
- **bbh_ste:**
  - objective: "A system prompt that makes the model find which type of error a German-to-English translation contains, answering with the option letter."
  - output_format: "Reply with the option letter in parentheses only, for example (A)."
  - seed:
    The following translations from German to English contain a particular error. That error will be one of the following types: Named Entities: An entity (names, places, locations, etc.) is changed to a different entity. Numerical Values: Numerical values (ordinals or cardinals), dates, and/or units are changed. Modifiers or Adjectives: The modifiers and adjectives pertaining to a noun are changed. Negation or Antonyms: Introduce or remove a negation or change comparatives to their antonyms. Facts: Trivial factual errors not pertaining to the above classes are introduced in the translations. Dropped Content: A significant clause in the translation is removed. Please identify that error. Options:
    (A) Modifiers or Adjectives
    (B) Numerical Values
    (C) Negation or Antonyms
    (D) Named Entities
    (E) Dropped Content
    (F) Facts
- **duration and size (Python code synthesis tasks):**
  - duration objective: "parse_duration(text) returns the length of a duration string in seconds, for the formats people and systems commonly write durations in, and raises ValueError for strings that are not valid durations."
  - size objective: "parse_size(text) returns the number of bytes in a data-size string, for the formats people and systems commonly write sizes in, and raises ValueError for strings that are not valid sizes."
  - Verbatim seed programs c_0:

```python
# duration seed c_0 (scores 0.458 on search, 0.417 on holdout)
def parse_duration(text):
    units = {"s": 1, "m": 60, "h": 3600}
    text = text.strip()
    return float(text[:-1]) * units[text[-1]]

# size seed c_0 (scores 0.208 on search, 0.208 on holdout)
def parse_size(text):
    units = {"B": 1, "K": 1024, "M": 1024**2, "G": 1024**3}
    text = text.strip()
    return int(text[:-1]) * units[text[-1]]
```

8.3 B.3 Verbatim Clustering and Judge Prompts

In `cluster` (Batch A.1), all failing cases of p_t are passed to one call of the clusterer model using `CLUSTER_PROMPT` in `benchmarks/evolution/factorial.py`:

```
Below are all the wrong answers a system prompt produced on its examples, one per
line. Group them into 3 to 7 failure patterns (fewer if there are fewer wrong
answers). For each pattern write one line: what goes wrong, how many of the wrong
answers it covers, and one representative wrong answer copied verbatim. Reply with
the patterns only.

{failures}
```

In `clustergold` (Batches A.2, A.3, A.4), `CLUSTER_GOLD_PROMPT` is used verbatim:

```
Below are all the wrong answers a system prompt produced on its examples, one per
line. Group them into 3 to 7 failure patterns (fewer if there are fewer wrong
answers). For each pattern write one line: what goes wrong, how many of the wrong
answers it covers, and one representative failure copied verbatim with its input,
the wrong answer and the expected answer in full. Reply with the patterns only.

{failures}
```

In `replies`, candidate outputs are scored at temperature 0 by `gpt-4.1-mini` using `JUDGE_PROMPT` in `benchmarks/evolution/replies_task.py`:

```
You check a customer support reply against requirements.

Requirements: {rubric}

Reply:
<<<
{reply}
>>>

Judge the meaning, not the wording. A requirement that the reply does not do
something is met when the reply does not do it; it need not say so.

Does the reply meet every requirement? Answer PASS or FAIL on the first line, then
one short sentence on why.
```

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael James Ryan, Meng Jiang, Christopher Potts, and Koushik Sen. Gepa: Reflective prompt evolution can outperform reinforcement learning. arXiv (Cornell University), 2025. URL http://arxiv.org/abs/2507.19457.

Farima Fatahi Bayat, Moin Aminnaseri, Pouya Pezeshkpour, and Estevam Hruschka. Reflective prompt tuning through language model function-calling. arXiv preprint, 2026. URL http://arxiv.org/abs/2605.21781v1.

Iñigo Casanueva, Tadas Temčinas, Daniela Gerz, Matthew Henderson, and Ivan Vulić. Efficient intent detection with dual sentence encoders. arXiv preprint, 2020. URL http://arxiv.org/abs/2003.04807v1.

Junyang Chen, Zecheng Wang, and Jingbang Chen. Adagepa: Adaptive feedback allocation for reflective prompt optimization. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2609.39927.

Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. arXiv (Cornell University), 2023. URL http://arxiv.org/abs/2309.16797.

Andrea Giummarra, Carmelo Baglieri, Antonio Di Maria, Andrea Farruggio, Michele Piatto, and Giovanni Micale. Understanding when reflective prompt and skill optimization helps on locally served reasoning models: A two-model study of gepa and skillopt. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2610.26020.

Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kai-Tao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. Evoprompt: Connecting llms with evolutionary algorithms yields powerful prompt optimizers, 2023. URL https://www.semanticscholar.org/paper/a9c75cf664f675a1b4034b0256ec3c5168e293df.

Derek Koh, Jinghui Mo, Benjamin H. Le, Jiening Zhan, Baofen Zheng, Kevin Bevis, Nathaniel C. Owen, Lauren Elizabeth Charney, Wenqiong Liu, and Jingwei Wu. Contrastive reflection for iterative prompt optimization. arXiv preprint, 2026. URL http://arxiv.org/abs/2606.30840v1.

Alexey Kravatskiy, Valentin Khrulkov, and Ivan Oseledets. Improvevolve: Basin-hopping meets llm-guided evolutionary search. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2602.10233.

Li-Hao Liu, Peng Tang, Kunwar Yashraj Singh, and Shabnam Ghadar. Espo: Error-structured prompt optimization via diagnose, diversify, and stabilize. arXiv (Cornell University), 2026. URL https://www.semanticscholar.org/paper/28eda5f75fa63a19fbd36d6540065ca806c0609a.

Elias Lumer, Sahil Sen, Vamse Kumar Subbiah, and Kevin Paul. Reflective prompt optimization of a deep-research agent: A topic-disjoint, multi-seed protocol for measuring transferable gains and prompt-level overfitting. arXiv preprint, 2026. URL http://arxiv.org/abs/2610.12601v1.

Ruotian Ma, Xiaolei Wang, Xin Zhou, Jian Li, Nan Du, Tao Gui, Qi Zhang, and Xuanjing Huang. Are large language models good prompt optimizers? arXiv preprint, 2024. URL http://arxiv.org/abs/2402.02101v1.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, S. Shirobokov, Borislav M. Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, Manish Kumar, and Abigail See. Alphaevolve: A coding agent for scientific and algorithmic discovery. arXiv (Cornell University), 2025. URL http://arxiv.org/abs/2506.13131.

Tal Oved, Roi Pony, Oshri Naparstek, and Udi Barzelay. Evolution or illusion? rethinking evaluation in llm evolutionary search. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2609.19799.

Reid Pryzant, Dan Iter, Jerry Zheng Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with "gradient descent" and beam search. *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. doi: 10.18653/v1/2023.emnlp-main.494. URL https://doi.org/10.18653/v1/2023.emnlp-main.494.

Yewen Pu, Zachery Miranda, Armando Solar-Lezama, and Leslie Pack Kaelbling. Selecting representative examples for program synthesis. arXiv preprint, 2017. URL http://arxiv.org/abs/1711.03243v3.

Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, Manish Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. Mathematical discoveries from program search with large language models. *Nature*, 2023. doi: 10.1038/s41586-023-06924-6. URL https://doi.org/10.1038/s41586-023-06924-6.

Jingyuan Selena She, Christopher Potts, Samuel R. Bowman, and Atticus Geiger. Scone: Benchmarking negation reasoning in language models with fine-tuning and in-context learning. *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023. doi: 10.18653/v1/2023.acl-short.154. URL https://doi.org/10.18653/v1/2023.acl-short.154.

Agamdeep Singh, Srishti Gautam, Priyanshu Gupta, Nikita Mehrotra, Tanmay Bakshi, and Sumit Gulwani. Coding agents are strong prompt optimizers. arXiv (Cornell University), 2026. URL https://arxiv.org/abs/2609.26261.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. arXiv preprint, 2022. URL http://arxiv.org/abs/2210.09261v1.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc Viet Le, Denny Zhou, and Xinyun Chen. Large language models as optimizers. arXiv (Cornell University), 2023. URL http://arxiv.org/abs/2309.03409.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic "differentiation" via text. arXiv preprint, 2024. URL http://arxiv.org/abs/2406.07496v1.

Xiaofan Zhou and Lu Cheng. Explore more, validate less: Efficient reflective prompt optimization. arXiv preprint, 2026. URL http://arxiv.org/abs/2610.27367v2.